Filtering Normalizing Flows

Otto Vintola

April 2, 2026

1 Introduction

Dynamical systems are the leading methodology for modeling a number of real-world phenomenoms. For example, the flagship problem in modern robotics, simultaneous localization and mapping (SLAM), involves estimating the latent state, often position, and constructing a map of the environment. Often SLAM is approached with traditional methods such as the Kalman Filter [1] and its variants [2, 3, 4, 5].

There exist other methods besides the Kalman filter, such as the particle filter [6]. However, the problem with the aforementioned methods are that they either assume the noise in the system to be Gaussian, or they become degenerate and/or computationally expensive [7, 8]. An easy way to overcome these issues is to abandon parametric assumptions of the filtering distribution.

The advances in deep generative modeling provides a promising direction to address these challenges. Particularly, normalizing flows (NF) provides a flexible method for constructing expressive probability distributions through sequenced invertible mappings [9, 10, 11]. NFs provide an efficient method for sampling and evaluating the likelihood which makes them especially suitable for modeling dynamical systems.

Filtering normalizing flows (FNFs) is an extension of the NF setting where the learned sequential invertible mappings approximate the filtering distribution commonly seen in Bayesian filtering and smoothing (BFS) [12]. Instead of assuming the filtering distribution to be Gaussian or representing it with particle weights, FNFs model it as a simple distribution that is transformed to a more complex one with an invertible mapping.

Moreover, FNFs extend to the smoothing perspective, where the goal is to estimate all of the latent states given all of the observations. By definition of NFs, the constructed mappings are invertible, thus they can be reversed to generalize to the entire trajectory.

2 Methods

In this section, the methodology is presented. First, a review of the Bayesian filtering problem for state-space models is presented, where the goal is to recover the distribution of the latent state given a set of observations. Then normalizing flows are introduced to construct the same distribution through a sequence of invertible transformations. Finally, NFs are presented in the Bayesian filtering framework.

2.1 Bayesian Filtering and Smoothing

In BFS, we consider a state-space model with latent states $\mathbf{x}_{0:T} = \{\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_T\}$ and noisy observations $\mathbf{y}_{1:T} = \{\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_T\}$. The system is typically defined through the following probabilistic model:

$$\mathbf{x}_k \sim p(\mathbf{x}_k \mid \mathbf{x}_{k-1}) \quad \text{(state transition model)}, \tag{1}$$

$$\mathbf{y}_k \sim p(\mathbf{y}_k \mid \mathbf{x}_k) \quad \text{(observation model)}. \tag{2}$$

The goal of Bayesian *filtering* is to recursively estimate the *filtering distribution*, i.e., the posterior distribution of the current latent state given all observations up to time k:

$$p(\mathbf{x}_k \mid \mathbf{y}_{1:k}), \quad k = 1, \ldots, T. \tag{3}$$

The filtering distribution can be computed iteratively with the following two-step procedure:

1. **Prediction step:**

$$p(\mathbf{x}_k \mid \mathbf{y}_{1:k-1}) = \int p(\mathbf{x}_k \mid \mathbf{x}_{k-1})\, p(\mathbf{x}_{k-1} \mid \mathbf{y}_{1:k-1})\, d\mathbf{x}_{k-1}, \tag{4}$$

2. **Update step:**

$$p(\mathbf{x}_k \mid \mathbf{y}_{1:k}) \propto p(\mathbf{y}_k \mid \mathbf{x}_k)\, p(\mathbf{x}_k \mid \mathbf{y}_{1:k-1}). \tag{5}$$

Another important goal is to compute the *smoothing distribution* distribution of the entire trajcectory of latent states given all of the measurements:

$$p(\mathbf{x}_{0:T} | \mathbf{y}_{1:T}), \text{ for } k = 1, \ldots, T. \tag{6}$$

Often when computing these distributions for non-linear and non-Gaussian systems, the integration becomes computationally intractable, which motivates the approximate methods such as particle filters [12].

2.2 Normalizing Flows

Normalizing flows (NFs) are a class of generative models that construct complex probability distributions through a sequence of invertible bijective transformations $f : \mathbb{R}^d \to \mathbb{R}^d$ [13, 11, 9, 10]. Given some base distribution $p_Z(\mathbf{z})$, a normalizing flow defines a new distribution $p_X(\mathbf{x})$ like so [14]:

$$\mathbf{x} = f(\mathbf{z}), \quad \mathbf{z} \sim p_Z(\mathbf{z}). \tag{7}$$

The change of variables formula allows us to compute the density of $\mathbf{x}$ as follows [13, 11, 9, 10]:

$$p_X(\mathbf{x}) = p_Z(f^{-1}(\mathbf{x})) \left| \det \left(\frac{\partial f^{-1}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|. \tag{8}$$

Normalizing flows can approximate any smooth distribution with a positive density given a sufficiently *powerful* transformation f [14]. This can be seen by considering the scenario where the base distribution is $p_Z \sim \text{Uniform}(0,1)^d$, and the transformation f is chosen to be the inverse Rosenblatt transformation [15]. The target distribution p_X is then exactly recovered by a sequence of conditional inverse CDFs as follows [14, **?**]:

$$\begin{aligned}
x_1 &= F^{-1}_{X_1}(z_1), \\
x_2 &= F^{-1}_{X_2|X_1}(z_2 \mid x_1), \\
x_3 &= F^{-1}_{X_3|X_1,X_2}(z_3 \mid x_1, x_2), \\
&\vdots \\
x_d &= F^{-1}_{X_d|X_{1:d-1}}(z_d \mid x_{1:d-1}).
\end{aligned} \tag{9}$$

where F_i^{-1} is the inverse CDF of the i-th dimension of the target distribution conditioned on the previous dimensions.

In practice, a single transformation f is often not powerful enough to capture the complexity of the target distribution, thus a sequence of transformations is used [9, 11, 10]:

$$\mathbf{x} = f_K \circ f_{K-1} \circ \cdots \circ f_1(\mathbf{z}). \tag{10}$$

The log density of $\mathbf{x}$ can be computed as the forward process—from the base distribution to the target distribution—as follows:

$$\log p_X(\mathbf{x}) = \log p_Z(\mathbf{z}) - \sum_{k=1}^{K} \log \left| \det \left(\frac{\partial f_k(\mathbf{h}_{k-1})}{\partial \mathbf{h}_{k-1}} \right) \right|, \tag{11}$$

where $\mathbf{h}_{k-1} = f_{k-1} \circ \cdots \circ f_1(\mathbf{z})$ is the intermediate representation after the k-th transformation. The log density calculation follows from the fact that the Jacobian of a composition of functions is the product of the Jacobians of the individual functions [14].

2.3 Filtering Normalizing Flows

While a normalizing flow could model any joint distribution $p(\mathbf{x}_{1:T})$, applying to the entire latent trajectory is computationally prohibitive and does not leverage the temporal dependencies in the system. A solution to this is presented in [13], where the authors propose to creatively constrain the expressiveness of the flow to match the recursive nature of Bayesian filtering. The flow is constrained to consist of a composition of functions $G = F_{T-1} \circ \cdots \circ F_1$, where each F_t only transforms a consecutive pair of latent states.

For the smoothing case, this is achieved by utilizing a transformation f parameterized by a weight matrix $\mathbf{A} \in \mathbb{R}^{dT \times dT}$ and bias $\mathbf{b} \in \mathbb{R}^{dT}$:

$$\mathbf{x}_{1:T} = f(\mathbf{z}_{1:T}; \mathbf{A}, \mathbf{b}) = \mathbf{z}_{1:T} + \mathbf{c} \odot h(\mathbf{A}^{-1}\mathbf{z}_{1:T} + \mathbf{b}), \tag{12}$$

where c is a context vector derived from observations, and $h(\cdot)$ is a smooth nonlinearity. To enforce temporal dependencies between consecutive states, the weight matrix $\mathbf{A}$ is constrained to be block-bi-diagonal:

$$\mathbf{A} = \begin{bmatrix} \mathbf{D}_1 & & & \\ \mathbf{L}_1 & \mathbf{D}_2 & & \\ & \ddots & \ddots & \\ & & \mathbf{L}_{T-1} & \mathbf{D}_T \end{bmatrix}. \tag{13}$$

Because $\mathbf{A}$ is lower-triangular, its determinant is simply the product of the determinants of the diagonal blocks $\mathbf{D}_t$, which allows the log density to be computed by:

$$\log q(\mathbf{x}_{1:T}) = \sum_{t=1}^{T} \log p(\mathbf{z}_t) + \sum_{t=1}^{T} \sum_{i=1}^{d} \log(1 + c_t(i) D_t(i,i) \psi_t(i)) - \sum_{t=1}^{T} \sum_{i=1}^{d} \log D_t(i,i), \tag{14}$$

where $\psi = h'(\mathbf{A}^{-1}\mathbf{z}_{1:T} + \mathbf{b})$ is the derivative of the nonlinearity. The log density calculation follows from Equation 9 and the determinant identity $\det(I + A^{-1}) = \det(A + I)/\det(A)$ [13].

3 Simulation

References

[1] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Transactions of the ASME–Journal of Basic Engineering*, vol. 82, no. Series D, pp. 35–45, 1960.

[2] M. Dorvash, A. Eslamian, and M. R. Ahmadzadeh, "Enhanced unscented kalman filter-based slam in dynamic environments: Euclidean approach," 2023.

[3] I. Ullah, X. Su, X. Zhang, and D. Choi, "Simultaneous localization and mapping based on kalman filter and extended kalman filter," *Wireless Communications and Mobile Computing*, vol. 2020, no. 1, p. 2138643, 2020.

[4] T. Sasaoka, I. Kimoto, Y. Kishimoto, K. Takaba, and H. Nakashima, "Multi-robot slam via information fusion extended kalman filters," *IFAC-PapersOnLine*, vol. 49, no. 22, pp. 303–308, 2016. 6th IFAC Workshop on Distributed Estimation and Control in Networked Systems NECSYS 2016.

[5] C. Yang and L. Xiang, *Distributed cooperative SLAM with adaptive Kalman filter and dynamic consensus*, pp. 183–191.

[6] N. J. Gordon, D. J. Salmond, and A. F. M. Smith, "Novel approach to nonlinear/non-gaussian bayesian state estimation," *IEE Proceedings F: Radar and Signal Processing*, vol. 140, no. 2, pp. 107–113, 1993.

[7] A. Doucet, N. de Freitas, and N. Gordon, "Sequential monte carlo methods in practice," *Statistics and Computing*, 2000.

[8] M. S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp, "A tutorial on particle filters for online nonlinear/non-gaussian bayesian tracking," *IEEE Transactions on Signal Processing*, vol. 50, no. 2, pp. 174–188, 2002.

[9] D. Rezende and S. Mohamed, "Variational inference with normalizing flows," in *Proceedings of the 32nd International Conference on Machine Learning*, vol. 37 of *Proceedings of Machine Learning Research*, pp. 1530–1538, PMLR, 2015.

[10] L. Dinh, J. Sohl-Dickstein, and S. Bengio, "Density estimation using real nvp," *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1605.08803.

[11] D. P. Kingma and P. Dhariwal, "Glow: Generative flow with invertible 1x1 convolutions," *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. arXiv:1807.03039.

[12] S. Särkkä and L. Svensson, *Bayesian Filtering and Smoothing*. Cambridge, UK: Cambridge University Press, second ed., 2023.

[13] H. S. Razaghi and L. Paninski, "Filtering normalizing flows," in *4th Workshop on Bayesian Deep Learning (NeurIPS 2019)*, (Vancouver, Canada), 2019. Workshop paper.

[14] K. P. Murphy, *Probabilistic Machine Learning: Advanced Topics*. Adaptive Computation and Machine Learning, Cambridge, Massachusetts: The MIT Press, 2023.

[15] M. Rosenblatt, "Remarks on a multivariate transformation," *The Annals of Mathematical Statistics*, vol. 23, pp. 470–472, September 1952.